Project Report: Offline Indexing Module for Image Database

Student: Dhia Eddine Merzougui
Subject: Image Indexing & Retrieval (Mini Project)
Date: 2025-11-29

1. Introduction

The objective of this project is to develop the Offline Module of a face recognition system. The goal of this module is to browse a dataset of facial images, preprocess them to remove lighting variations, extract robust texture features using the Local Ternary Patterns (LTP) method, and store these features in a structured database (Index) for later retrieval. The dataset consists of 400 images (40 distinct individuals, 10 images per person).

2. Methodology & Development Process

The development was divided into four distinct stages: Directory Browsing, Image Preprocessing, Feature Extraction, and Indexing.

2.1 Directory Browsing & Organization

The input dataset contained images with filenames encoding the person's identity (e.g., *1.pgms1.jpg* indicates Person 1). I utilized Python's *pathlib* and *glob* to iterate through the raw image folder. We implemented a Regex (Regular Expression) pattern *^(\d+)\.pgms(\d+)\.jpg$* to accurately extract the Person ID from the filenames. The system automatically creates a structured output directory (*faces_preprocessed/personX/*) to organize the intermediate results.

2.2 Image Preprocessing (Tan & Triggs Chain)

Before feature extraction, it is crucial to normalize the images to make the recognition robust against lighting changes and shadows. We implemented the preprocessing chain recommended by Tan & Triggs

• **Gamma Correction (gamma = 0.2):** We applied a non-linear transformation to enhance details in dark regions (shadows). A Gamma value of 0.2 was chosen to aggressively reduce the effect of heavy side-lighting.

• **Difference of Gaussians (DoG)**: We applied band-pass filtering to remove high-frequency noise and low-frequency shading. We calculated two Gaussian blurs (sigma1=1.0 and sigma2=2.0) and subtracted them. This results in a 'grey' edge-map image where lighting gradients are removed.

• **Masking**: A border mask of 5 pixels was applied to remove artifacts at the image edges caused by the convolution process.

• **Contrast Equalization**: We normalized the pixel intensities using a robust mean calculation to ensure the final image contrast is consistent across the entire dataset.

2.3 Feature Extraction: Local Ternary Patterns (LTP)

Unlike Local Binary Patterns (LBP), which are sensitive to noise in uniform regions, LTP introduces a tolerance threshold *t* (set to 5 in our code).

The Algorithm: We iterated through every pixel of the preprocessed image. For each center pixel *c*, we compared it to its 8 neighbors *p*:
• If p >= c + t: The neighbor is considered **1**.
• If p <= c - t: The neighbor is considered **-1**.
• Otherwise: The neighbor is **0**.

Splitting: Since the resulting ternary code is difficult to process directly, we split the signal into two binary matrices: Upper Matrix (captures positive patterns/1s) and Lower Matrix (captures negative patterns/-1s).

Histogram Generation: We calculated the histogram for the Upper matrix (256 bins) and the Lower matrix (256 bins). These were normalized to make the features scale-invariant and concatenated to form a final **512-dimensional feature vector** for each image.

2.4 File Structure Design (Indexing)

To store the extracted features efficiently for the Online Module, we designed a key-value store using the **JSON** format.
• **Format:** JSON (index.json).
• **Key:** The relative file path (e.g., 'person1/1.pgms1.jpg'), serving as a unique identifier.
• **Value:** The 512-point list of floating-point numbers representing the LTP histogram.

Advantages of this structure:
1. **Human-Readable:** We can easily verify if the features are being calculated (non-zero values) by opening the text file.
1. **Portability:** JSON is language-independent, allowing the search engine to be implemented in any language.
1. **Speed:** For a dataset of 400 images, loading this JSON file into a Python dictionary is near-instantaneous (O(1) access time).

3. Implementation Details

The solution was implemented in **Python 3** using the following libraries:
• **OpenCV (cv2):** Used for efficient image loading, Gamma LUT (Look-Up Table) application, and Gaussian blurring.
• **NumPy:** Used for matrix operations, histogram calculation, and vector concatenation.
• **Re (Regex):** Used for robust filename parsing.

Note on Loops: While Python libraries offer built-in LBP functions, we manually implemented the LTP neighbor-comparison loops to demonstrate a clear understanding of the underlying algorithmic logic.

4. Conclusion

The developed Offline Module successfully processes the raw dataset and generates a robust index. The preprocessing chain effectively neutralizes lighting variations (as evidenced by the resulting texture-focused grey images), and the LTP algorithm captures detailed facial features.

The resulting *index.json* provides a lightweight and structured foundation for the subsequent image retrieval/matching task.